pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Recurrent Networks: LSTM and GRU

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Where day 11 left the curve

The question, in the past tense

For each date in the development years: the 10 Yr change in bp, from a model fitted only on earlier dates, using the 20 changes before it.

3.21

zero change (bp)

3.22

training mean (bp)

3.35

day 11's dense network (bp)

Today: layers that read the window in order, one day at a time, and carry a state.

A walk-forward error on past dates is not evidence about any later date, and not a view on rates.

What the output may be called

Allowed names

fitted change in bp

walk-forward error

the last value carried forward

Never called

a forecast or a prediction

tomorrow's move or a call

an outlook or a view on rates

Development years: 2,001 dates, 2015-01-02 to 2022-12-30. The held-out years open once, on day 14.

How a recurrent layer works

1. A state

a small vector, zero before day 1

2. Each day

mix the day's input with yesterday's state

3. Squash

tanh, kept as the new state

4. Output

the state after the last day

You control

The number of units and the window.

What comes out

One vector of u numbers per window.

A recurrent step, carried along the window

```
h = zeros(16)                          # before day 1
h = tanh(x[t] * W + h @ U + b)         # for each day t of the window
```

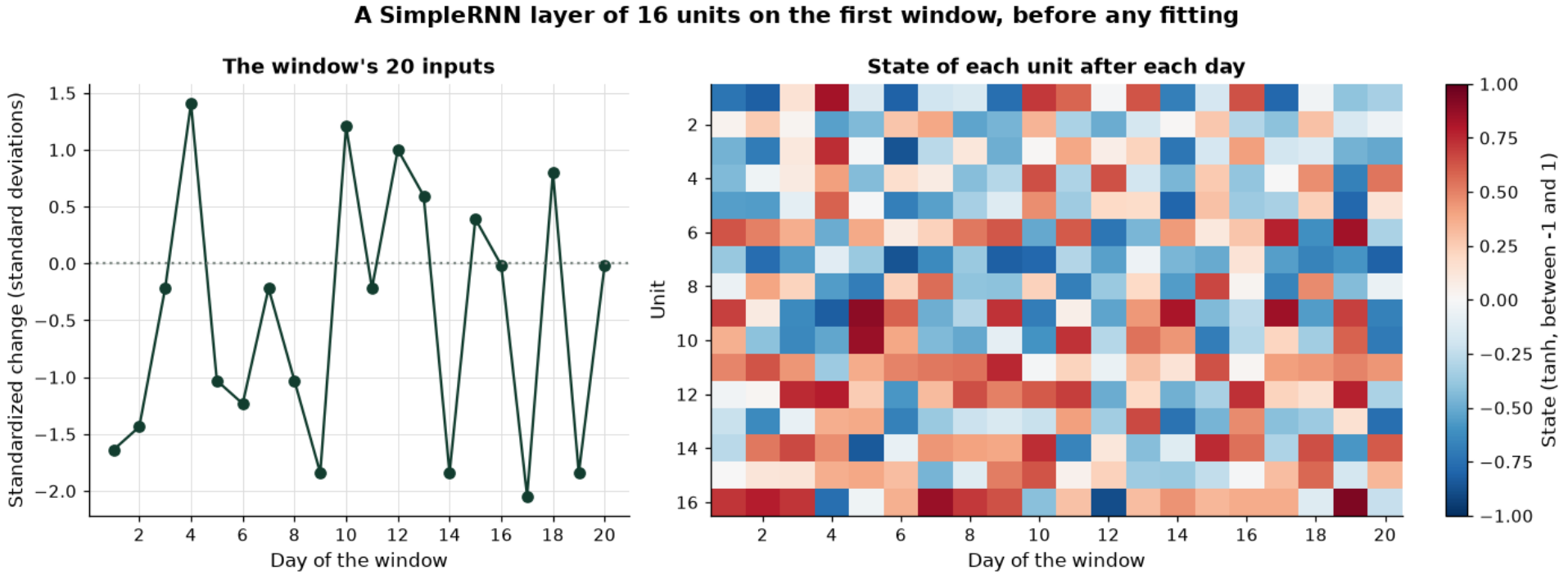
Weight	Shape	What it does
W, the kernel	(1, 16)	today's input, one weight per unit
U, the recurrent kernel	(16, 16)	yesterday's state into today's
b, the bias	(16,)	an intercept per unit

Against Keras

Day 20 state within 1e-6: True

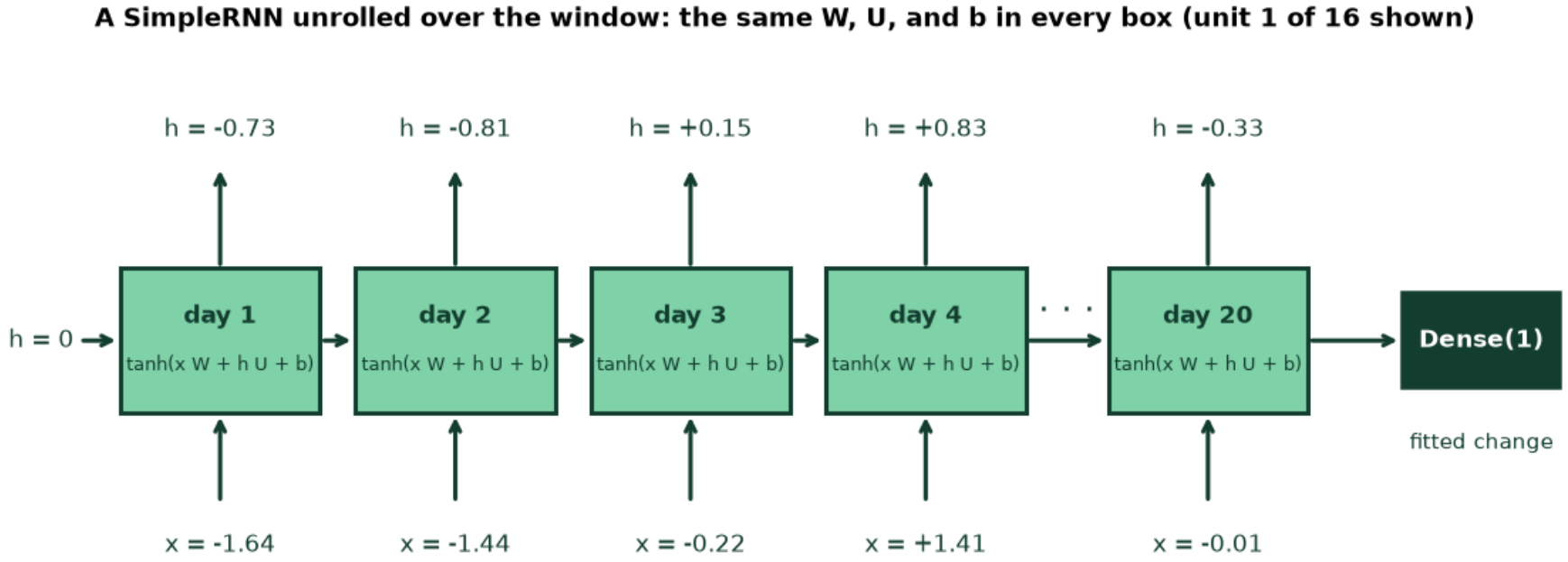
First window 2015-01-05 to 2015-02-02, scaled with fold 1's scaler (mean 0.0707 bp, sd 4.9244 bp); its target 2015-02-03 is not in it.

The states, day by day



Columns are days, rows are units; red is a positive state, blue a negative one. All between -1 and 1 because tanh is.

The layer unrolled: one box per day



One box's worth of weights, not twenty: the boxes are the same layer on different days.

Counting a recurrent layer's weights

```
SimpleRNN  u (u + f + 1)
LSTM       4 u (u + f + 1)
GRU        3 u (u + f + 2)
```

Layer, 16 units	1 channel	11 channels
SimpleRNN	288	448
LSTM	1,152	1,792
GRU	912	1,392

The window's length is in no formula

The same weights are used on every day, so 20 days and 10 days need the same count. Day 11's dense layer had a weight per day.

Formula equal to count_params in every row; 53 tests pass by the end of the day.

How an LSTM works

1. Cell state

a second vector, changed only through gates

2. Forget gate

how much of yesterday's cell to keep

3. Input gate

how much of today's candidate to write

4. Output gate

how much of the cell to show

You control

The number of units; the gates' weights are fitted.

What comes out

The final hidden state, one vector per window.

The LSTM's gates

Input gate

sigmoid: how much to write

unit 1, day 1: 0.4118

Forget gate

sigmoid: how much to keep

unit 1, day 1: 0.8145

Candidate

tanh: what today would write

unit 1, day 1: 0.3701

Output gate

sigmoid: how much to show

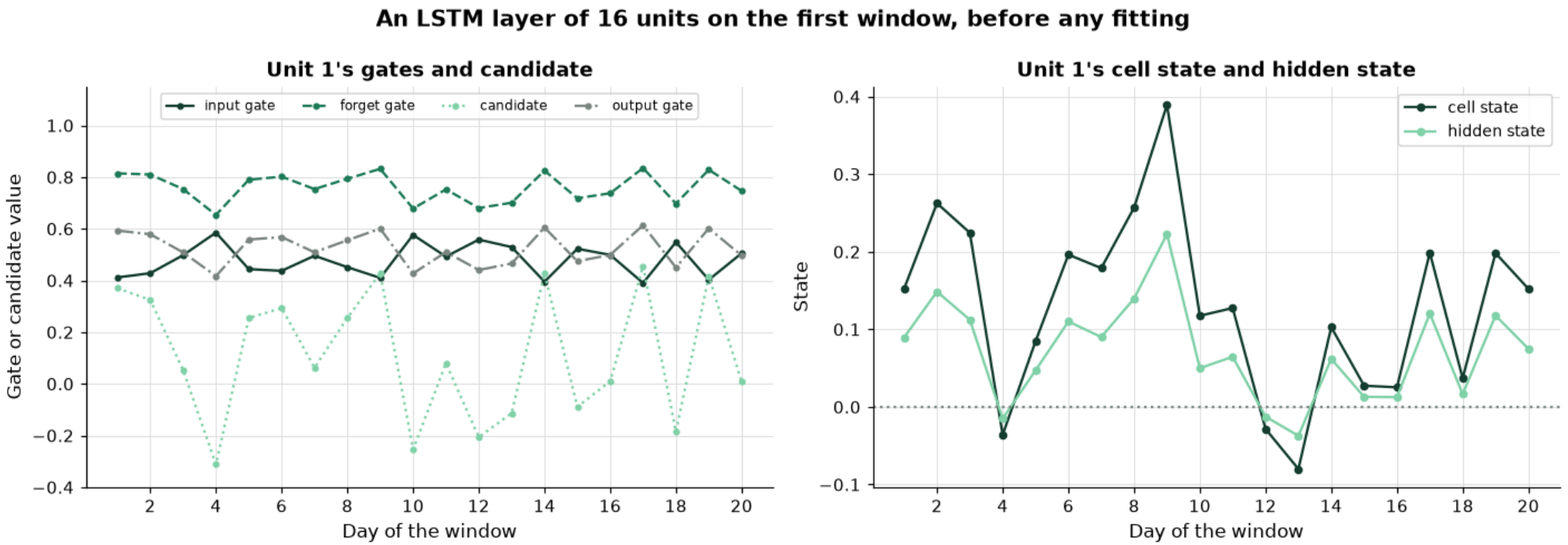
unit 1, day 1: 0.5928

```
c = forget * c + input * candidate
h = output * tanh(c)
```

Weights four times as wide: (1, 64), (16, 64), (64,). Forget intercepts start at 1.

lstm_step carried 20 days equals the Keras layer within 1e-6: True.

One unit's gates, day by day



Starting weights, nothing fitted. The hidden state is never larger in size than the cell state.

The name on the layer

Sepp Hochreiter and Juergen Schmidhuber, 1 November 1997

"Long Short-Term Memory"

Neural Computation, volume 9, issue 8, pages 1735 to 1780

keras.layers.LSTM: the cell state a step can keep or forget through its gates.

Source, read 6 October 2026: the Crossref record of the paper (DOI 10.1162/neco.1997.9.8.1735). Bibliographic facts only; the paper itself was not read for this course.

How a GRU works

1. One state

no separate cell state

2. Update gate

keep yesterday's, or write the candidate

3. Reset gate

how much of yesterday the candidate sees

4. Fewer weights

three blocks instead of four

You control

The number of units; the three blocks are fitted.

What comes out

The final state, one vector per window.

The GRU: one state, two gates

Update gate

Mixes yesterday's state with today's candidate.

Reset gate

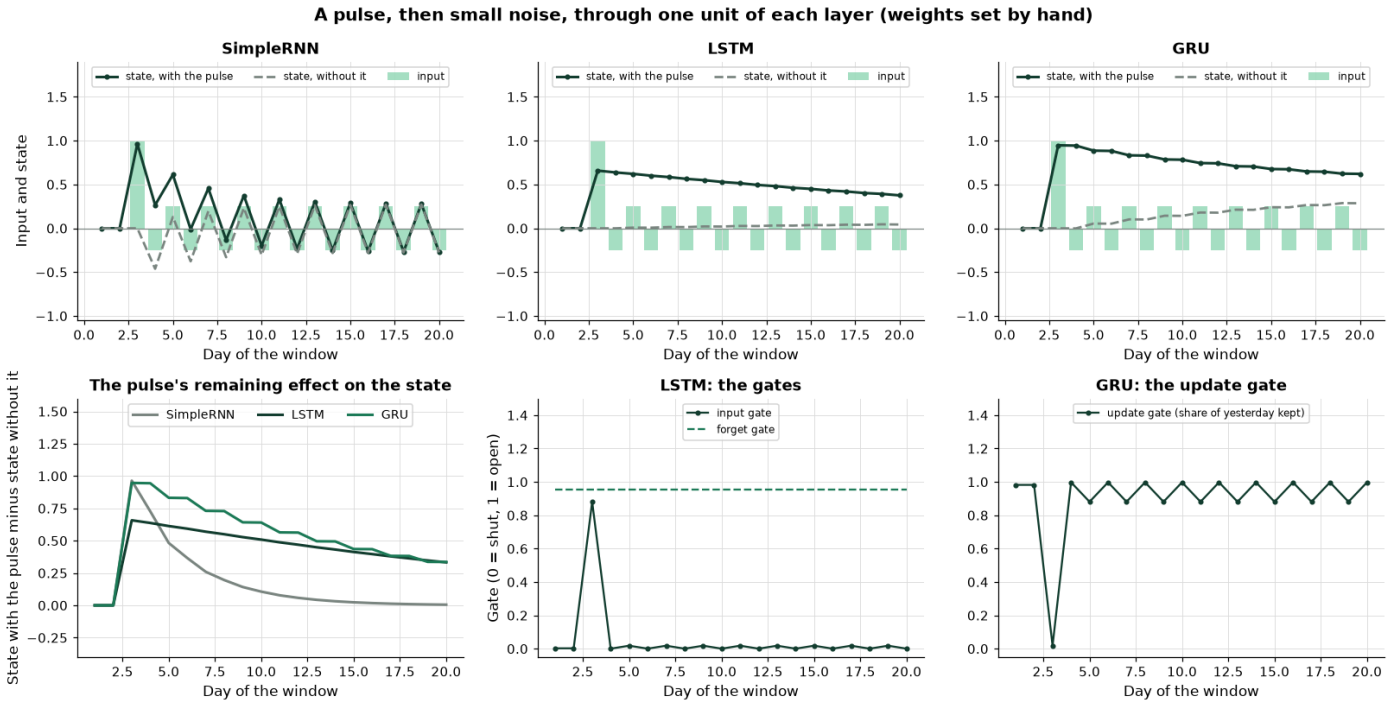
Sets how much of yesterday's state goes into the candidate.

Kernel (1, 48), recurrent kernel (16, 48), bias (2, 48): 912 weights on one channel.

Two bias rows (reset_after=True, Keras's default): the + 2 in 3 u (u + f + 2).

Usually traced to Cho and six co-authors, arXiv 1406.1078, first version 3 June 2014 (the arXiv record, read 6 October 2026; the paper was not read for this course).

What the gates do: a pulse, then small noise



Weights set by hand to show what the gates can do. The SimpleRNN forgets the pulse; the gated layers keep it and let the noise in only a little.

Every setting of the three recurrent layers

Setting	SimpleRNN	LSTM	GRU	What it controls
units	required	required	required	size of the state; tune first
activation	'tanh'	'tanh'	'tanh'	squashing of candidate and state
recurrent_activation	-	'sigmoid'	'sigmoid'	squashing of the gates
unit_forget_bias	-	True	-	forget gate starts at 1 (LSTM)
reset_after	-	-	True	reset gate after the multiply (GRU)
dropout	0.0	0.0	0.0	share of inputs dropped; tune first
return_sequences	False	False	False	a state for every day, to stack layers

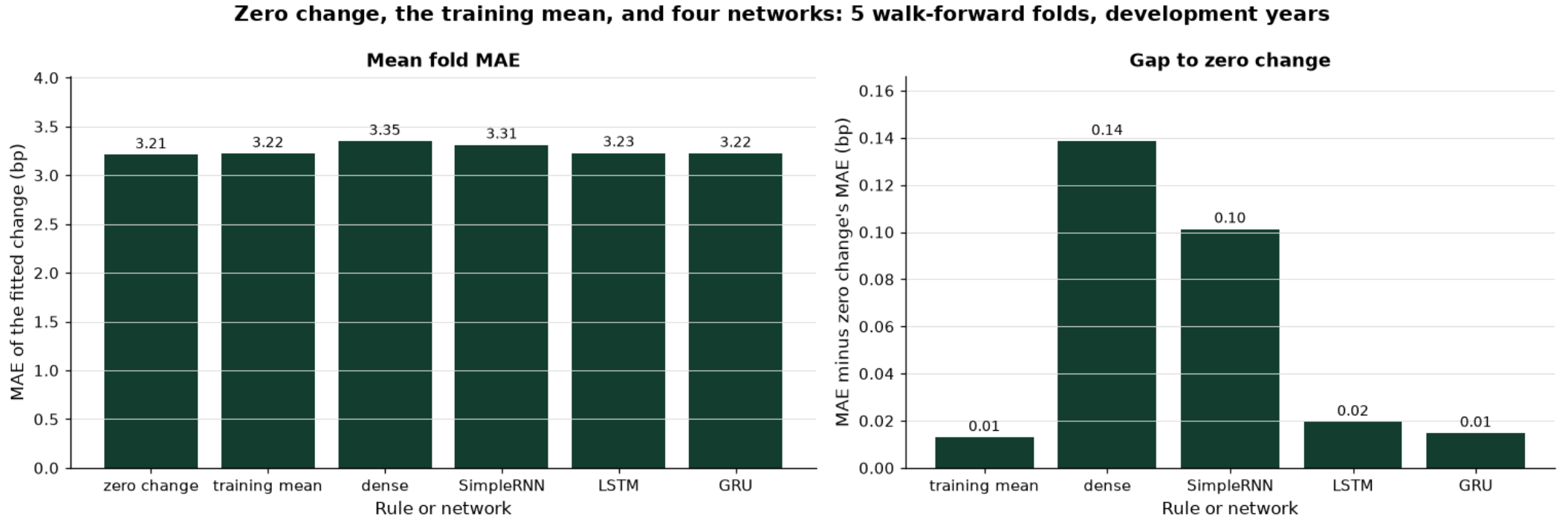
25 settings between the three layers; the notebook's table lists every one, with what raising and lowering it does.

Walk-forward: three recurrent networks

Rule or network	Mean fold MAE (bp)	Folds below zero change
zero change (last value carried forward)	3.21	
training mean	3.22	0 of 5
dense (day 11's, rebuilt)	3.35	0 of 5
SimpleRNN	3.31	0 of 5
LSTM	3.23	0 of 5
GRU	3.22	1 of 5

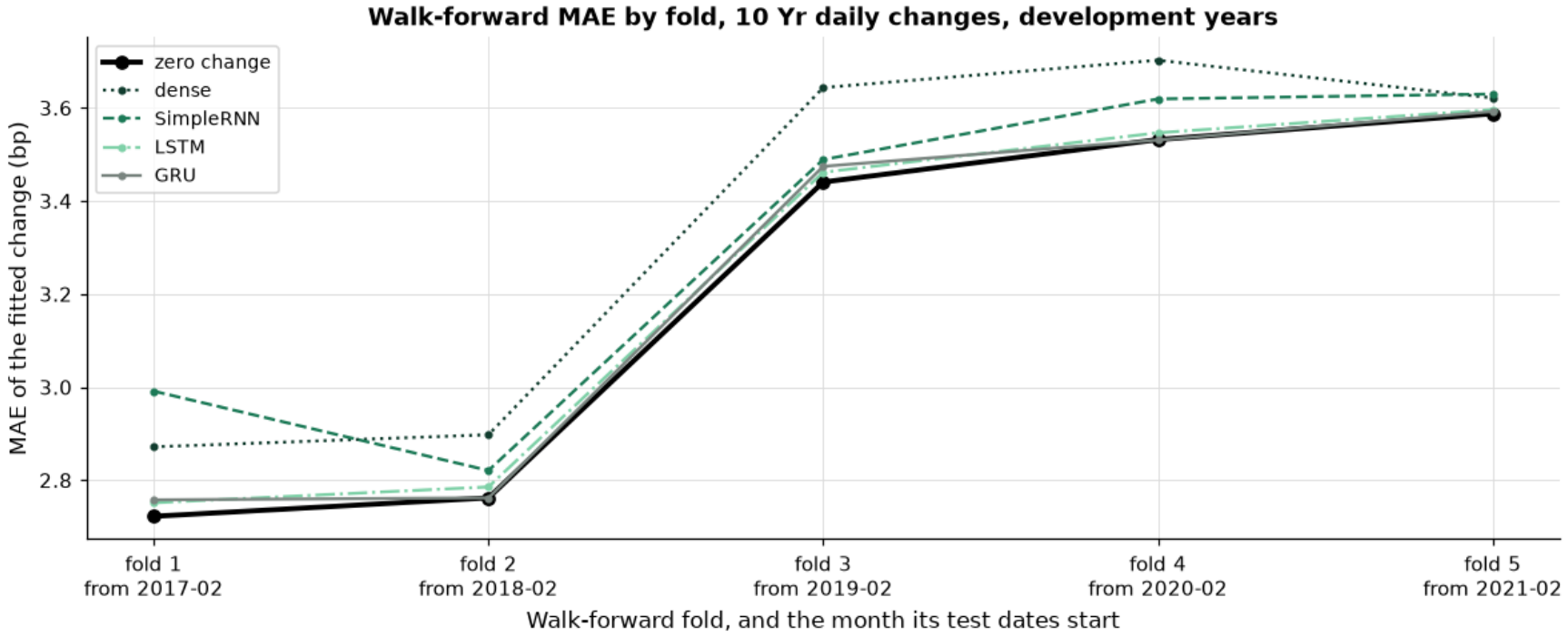
Every network scored more than zero change. The GRU, closest, is chosen on the development years.

The six scores as bars



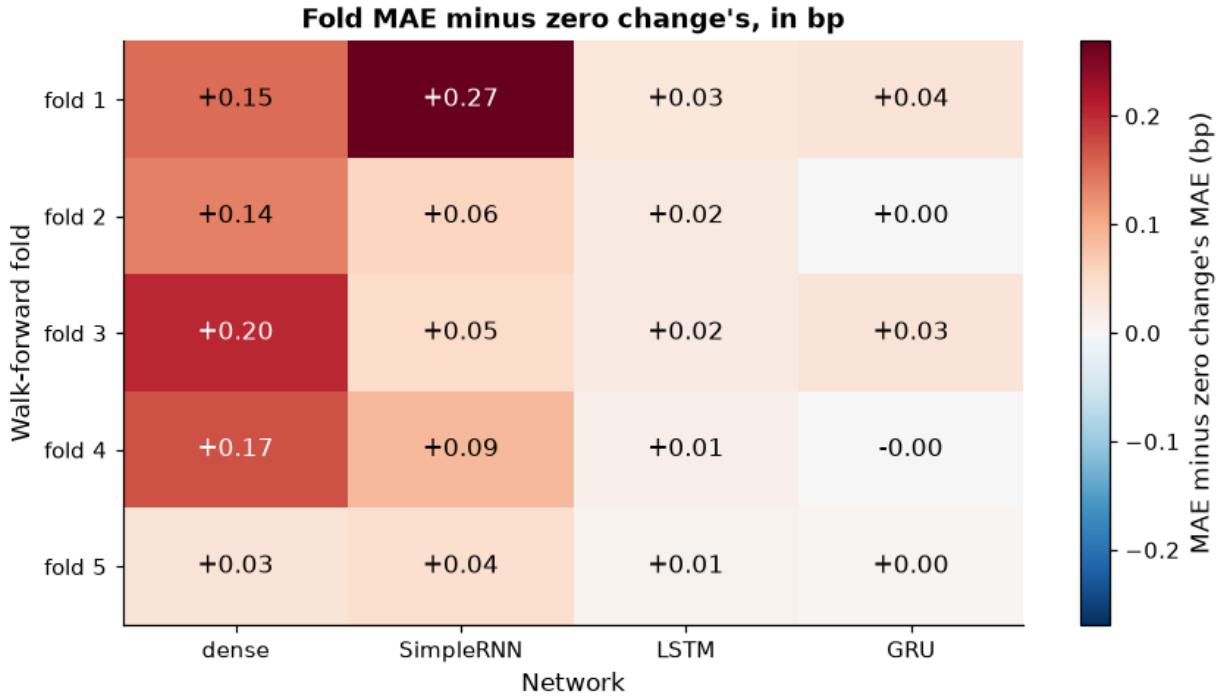
Zero change, 3.21 bp, is the lowest bar; every other bar sits above it.

Fold by fold: beside zero change



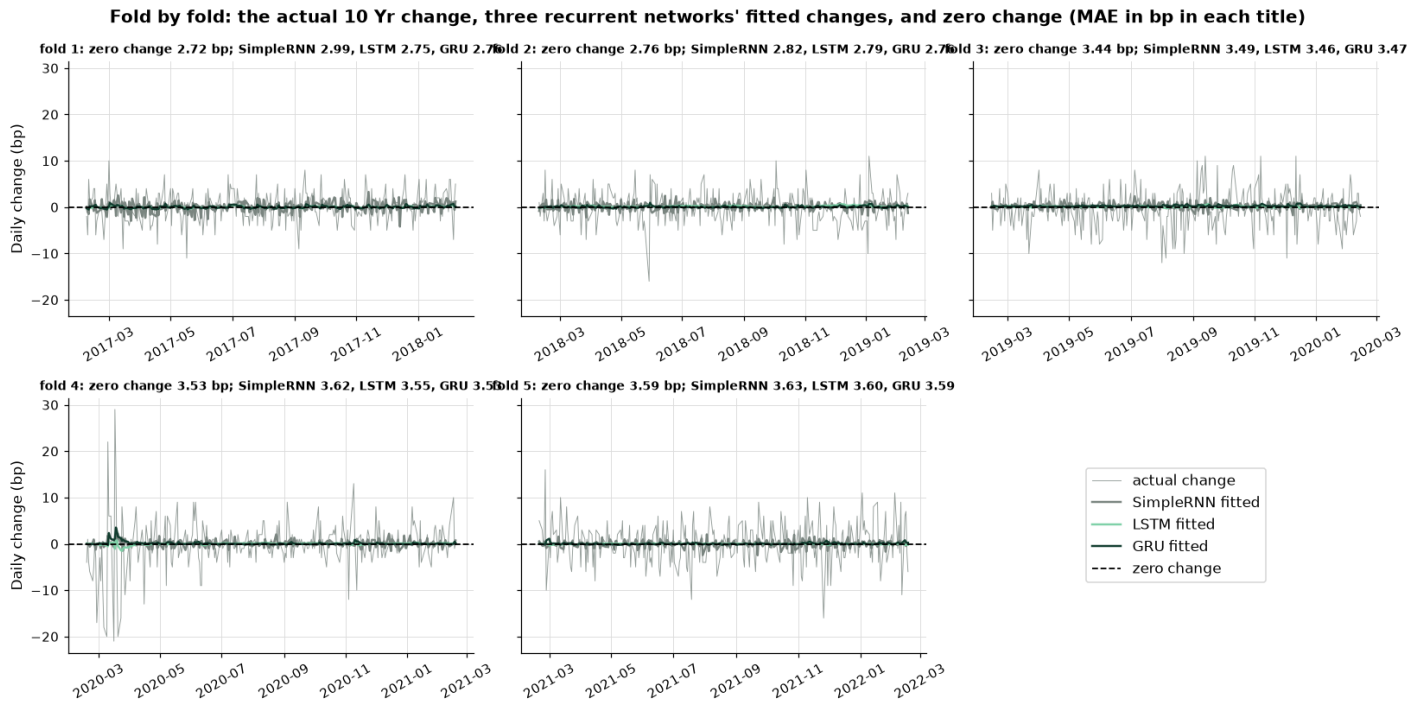
Folds below zero change: dense 0 of 5, SimpleRNN 0 of 5, LSTM 0 of 5, GRU 1 of 5. Every line moves with the fold.

Every fold and network: the gaps



Red is above zero change, blue below. Pale cells are within a few hundredths of a bp.

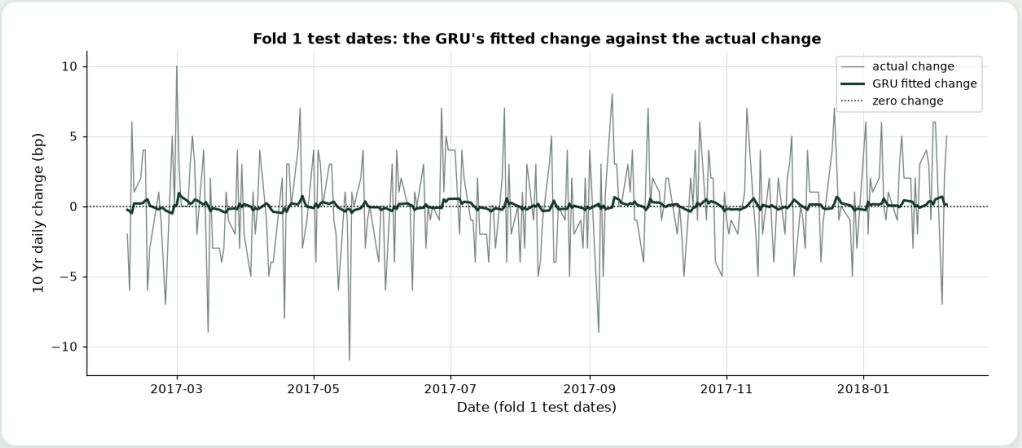
Fitted against actual, fold by fold, three networks



In every fold the three fitted lines hug zero while the actual change swings; the scores sit within a few hundredths of a bp of zero change's.

Why a network ends near the mean

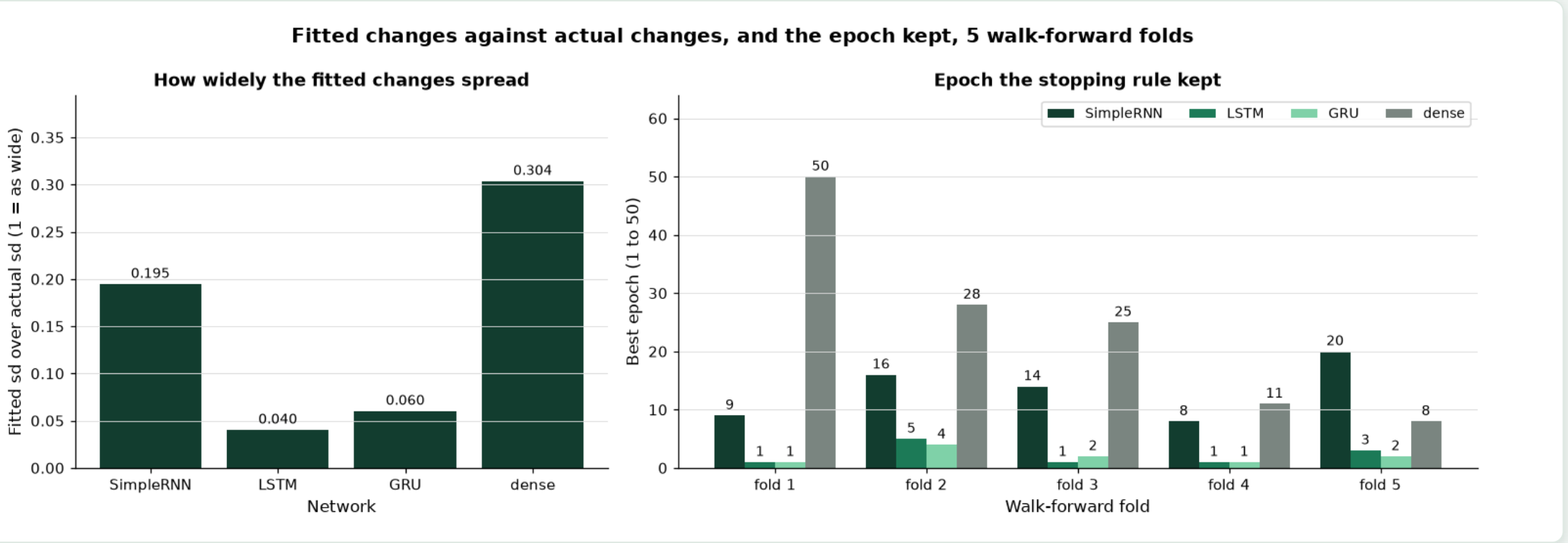
Network	Spread ratio	Best epochs by fold
dense	0.3037	[50, 28, 25, 11, 8]
SimpleRNN	0.1947	[9, 16, 14, 8, 20]
LSTM	0.0404	[1, 5, 1, 1, 3]
GRU	0.0603	[1, 4, 2, 1, 2]



Fitted sd over actual sd (4.3254 bp). 0 would be one number on every date; best epoch 1 is a network that barely moved.

Closest to zero change by fitting little; the networks that fitted more scored worse.

Spread and epochs kept



A short bar on the left and a bar of height 1 on the right are the same story: a network that barely moved from its start.

One setting changed: the window

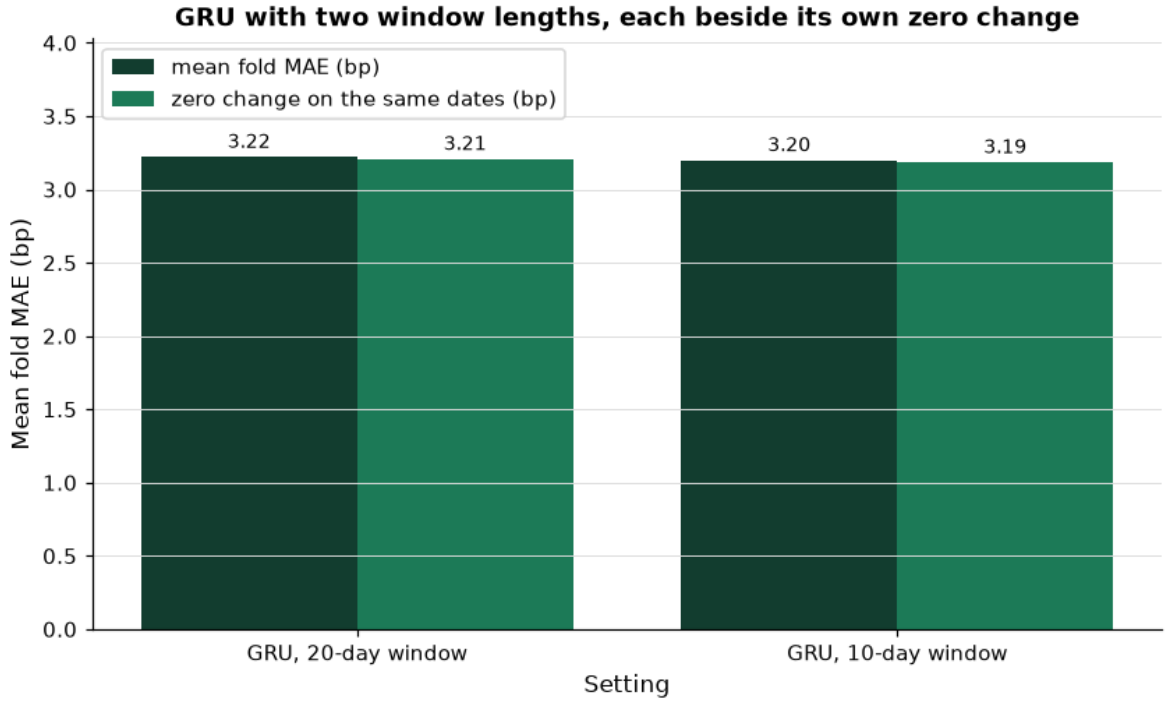
GRU	MAE (bp)	Zero change (bp)	Folds below	First test date
20-day window	3.22	3.21	1 of 5	2017-02-07
10-day window	3.20	3.19	1 of 5	2017-01-24

Each window against its own zero change

A shorter window starts its targets earlier, so its folds hold slightly different dates.

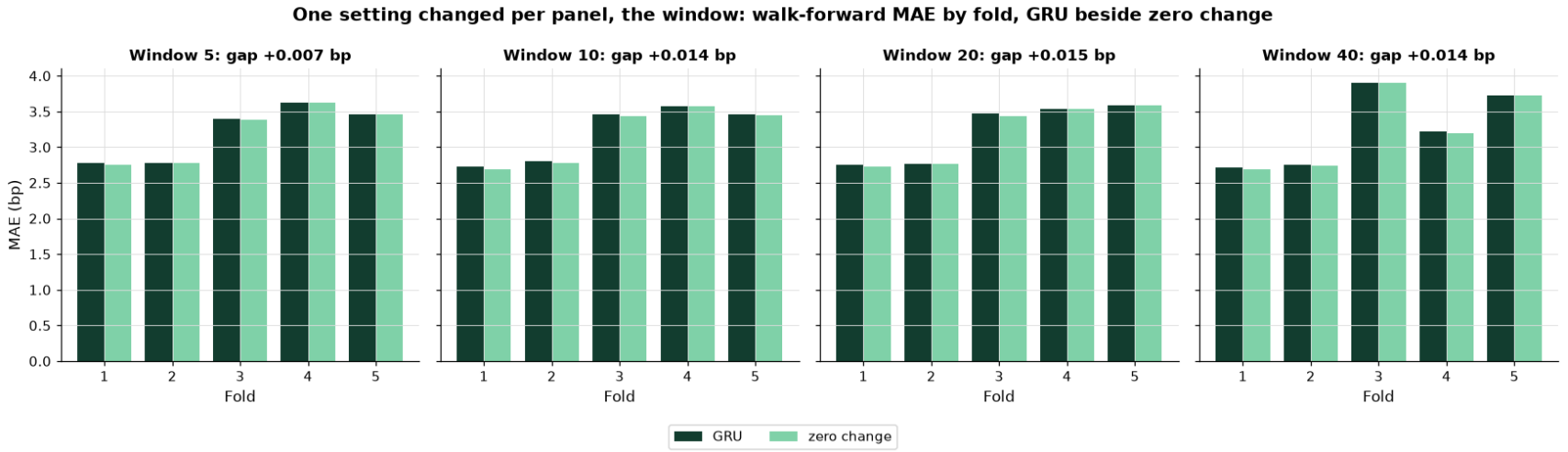
One setting changed from the GRU: the window, 20 to 10 trading days; chosen on the development years' five folds.

Two windows as bars



The zero-change bars differ a little: the shorter window starts its targets earlier. Each is compared with its own.

The window, four settings, one per panel



Window	Network	Zero change	Gap	Folds below
5	3.206	3.199	0.007	0
10	3.200	3.187	0.014	1
20	3.223	3.209	0.015	1
40	3.263	3.249	0.014	0

Excel side by side

Excel side by side (tested in Excel)

`=TANH($B3*rw!$A$1:$P$1+MMULT(C2:R2,rw!$A$3:$P$18)+rw!$A$2:$P$2)`	SimpleRNN, 16 units, spilled; filled down 20 days
`=TANH($B4*rw!A1+SUMPRODUCT(C3:R3,TRANSPOSE(rw!A3:A18))+rw!A2)`	one unit, by SUMPRODUCT
`=1/(1+EXP(-AY3:BN3))*S2:AH2+1/(1+EXP(-AI3:AX3))*TANH(BO3:CD3)`	LSTM cell state (forget, input, candidate)
`=1/(1+EXP(-CE3:CT3))*TANH(S3:AH3)`	LSTM hidden state (output gate)
`=AVERAGE(ABS(B2:B253-C2:C253))`	GRU MAE, fold 1: 2.7576 bp
`=STDEV.P(C2:C1261)/STDEV.P(B2:B1261)`	spread ratio 0.0603

No recurrent layer, training, or walk-forward in Excel. Every number equals Python's within 1e-12.

Save the day: four commits

1. Morning

netkit as day 11 left it

2. Code

two functions, five tests

3. Day 11's log

rows on changes, rebuilt

4. Today's rows

four networks added

```
model,channels,window,rows,folds,mae_bp,zero_change_mae_bp,folds_below_zero_change
zero_change,1,20,development,5,3.2087,3.2087,0
training_mean,1,20,development,5,3.2216,3.2087,0
dense_16_changes,1,20,development,5,3.3472,3.2087,0
simple_rnn_16,1,20,development,5,3.3099,3.2087,0
lstm_16,1,20,development,5,3.2281,3.2087,0
gru_16,1,20,development,5,3.2234,3.2087,1
gru_16_window_10,1,10,development,5,3.2003,3.1865,1
```

git show --stat HEAD: 1 file changed, 4 insertions(+). Added rows only.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: one scaler fitted on every development row: fold 1's mean 0.0880 bp, not 0.0707 bp.

Code that runs is not the same as code that is right.

What you can do now

- Carry a recurrent step along a window by hand
- Count a recurrent layer's weights
- Match an LSTM with lstm_step, gate by gate
- Score SimpleRNN, LSTM, GRU against zero change
- Read a spread ratio and the best epochs

Week 3, Day 3 One-dimensional convolutions and many tenors